HeavyHeader

C/C++ 构建效能治理平台

把「删掉这行 include 会怎样」变成一次可复现的受控实验

第四届开放原子大赛 · 2026 开源行业解决方案创新赛
赛道一 · 开源基础软件与解决方案

项目负责人：王子周
申报主体类型：开源社区（个人）
精简评审版
数据冻结：2026 年 10 月 22 日

代码仓库：`https://github.com/wzzrambler/heavyheader`
本版只保留评审决策需要的论证；配套完整版（含全部逐轮实验记录）见同目录《HeavyHeader-作品介绍文档.pdf》。两版数字完全一致，章节对应关系见仓库 `docs/restructure-map.md`。

Apache-2.0 · 本作品以开源方式提交

一、项目概览与核心结论

1.1 三个核心主张

本项目与同类工具的三点核心差异，以及每一点的证据位置。

	一句话	证据在哪
① 问题不是找出昂贵的头文件	而是证明删掉某一条 include 是否真的有净收益——反事实构建实验（CBE）与独占闭包把「谁贵」变成「删了会怎样」	2.3 / 2.4（方法）；2.5（消融）；3.2–3.3（收益）
② 只编译通过不够	必须防止性能数字来自被破坏的程序：spdlog −77.8% 的大额「提升」经汇编指纹比对是删掉了函数定义；汇编闸门 + 强制复测把这类假收益拦下	2.3（三真值）；3.4（反例）；3.5（三档证据）
③ 这不是一次性分析工具	而是一条可复现、可审查、可接入 CI 的治理链路：上游 PR、门禁、成本考古、语义化发布与 759 个测试	3.3（上游 PR）；4.2（成本考古）；4.5（门禁与 CI）

1.2 一条主线：从成本热点到 CI 门禁

十四个命令不是十四个卖点，是同一条流水线上的六个环节。每一步都对应仓库里可复现的实验数据。

环节	命令	这一步发生什么	实测证据
① 发现成本热点	`scan`	真实编译器自述解析行为（-H / -ftime-report），归因到具体 include 行	nlohmann/json 25 TU：487 万行解析量归因到 9,294 条 include 边
② 提出删除候选	`scan`	只提「删掉这一行能少解析什么」的候选，按独占闭包与用法判定排序	四语料 100 个 TU：静态筛选出 26 条疑似未使用候选，未出现所定义的假阳性（不代表通过真实编译验证，更不代表语义等价）
③ 真实编译器验证	`verify`	真的注释掉那一条 include 再编：编译状态、少解析几个文件、汇编指纹是否变化（第三条真值）	spdlog：−77.8% 的大额「提升」经汇编比对是删掉了公共 API 的实现，被拦下（§3.4）
④ 重复测量收益	`verify`	交替 A/B、min-of-3、越过噪声线才算数，improved 必须复测	doctest −3.01%、zlib −2.125%、zstd −46.5%，均复测通过
⑤ 生成可审查补丁	`pr-comment`	把结论、证据与具体改动的 diff 变成人可以评审的 PR 评论	已提交：doctest#1212、madler/zlib#1344 两条上游 PR（§3.3，均 open、未合并）
⑥ 成本回归进 CI	`gate`	以证实过的基线为尺子，每次提交超阈即报——治理是持续的，不是一次性的	注入回归 3/3 检出、无回归 3/3 通过、构建损坏 3/3 拦截（G3）

链条读法：环节①②竞品也能做；③④⑤⑥是本项目的壁垒——尤其是③的第三条真值（汇编指纹）与⑥的持续治理。前者让「编得过」不再等于「代码生成一致」，后者把一次性分析变成工程纪律。

1.3 头条数字与五组硬证据

两个头条数字，建议倒序读：−77.8% 是这个项目存在的理由——它是一个被拦下来的假优化；−11.4% 是能量化的收益，但它来自 PCH 结构性改动，不是删 include，删 include 的端到端成绩是 222 条边里确认 5 条（2.3%）。两个数字口径完全不同，全文一共三类收益口径，定义见 3.1。

头条数字	它是什么指标（引用时必须带口径）	出处
−77.8%	单条 include 的计时变化，而且是一个假收益：spdlog cfg.cpp 566 → 125 ms，三次独立运行复现。接上第三条真值后发现汇编从 10,329 行塌缩到 40 行——删掉的是函数定义，`-fsyntax-only` 照样过、链接期才炸。这个数字的价值在于被识破：没有第三轴，它会被当成收益上报	3.4
−11.4%	完整构建的编译 CPU 降幅：292 → 259 s，nlohmann/json 63 个编译单元，63 个测试套件 4,264,414 条断言逐个一致。它来自 PCH 结构性改动，不是删一条 include 得到的；删 include 的端到端成绩是 222 条边里确认 5 条（2.3%）	3.2

五组证据分别对应：方法有必要、收益可量化、能识别看起来很成功的错误优化、已提交上游 PR、治理能力已实现。后两组强度不如前三组，此处如实标注——上游 PR 是已发生的事实，门禁是在自己注入的回归上自测的。

#	证据	关键数字	本版章节
1	方法有必要：消融实验——拿掉任何一层，都有实测记录在案的错误结论	拿掉独占闭包：前排被所有单元共用的头占据，删掉必然编译失败；拿掉交替测量：2.005% 的机器漂移被整块报成优化；拿掉强制复测：第一轮 4 条 improved 复测 0/4 复现	2.5
2	收益可量化：完整构建与单条 include 分开口径	完整构建编译 CPU 292 → 259 s（−11.4%，PCH 改动）；单条 include：doctest −3.01%、zlib −2.125%、zstd −46.5%；端到端分母：222 条边里确认 5 条（2.3%）	3.2 / 3.3
3	能识别「看起来很成功」的错误优化	spdlog 删 cfg.cpp 一条 include：计时 −77.8%，但汇编从 10,329 行塌缩到 40 行（删掉了函数定义），被闸门拦下	3.4
4	已提交上游 PR（事实陈述，不是「已被采用」）	doctest#1212 有一条维护者的礼貌回应，codecov 报告改动行被测试全覆盖；madler/zlib#1344 无任何回应。两条均为 open、未被合并	3.3
5	治理能力已实现（自测充分，尚无第三方使用证据）	G3 门禁在自己注入的回归上 3/3 检出、无回归 3/3 放行、构建损坏 3/3 拦截；CI 14 岗位全绿（2026-10-22 核实，含 macOS）；759 个测试；零第三方依赖（由测试守护）	4.5

产品的定位：它不承诺替你找到优化，它承诺把「看起来能省」变成「确认能省」或「确认不能省」，并把这个结论固化成能拦住回归的基线。关键词：构建效能；头文件成本归因；include 治理；CI 门禁。

二、创新方法与竞品边界

2.1 问题不是「谁贵」，而是「删了会怎样」

一个 C++ 项目的预处理与解析成本，一阶近似正比于「编译器被迫读了多少头文件文本」：一个 4700 行的头被 3000 个编译单元引用时，项目要为它解析 1410 万行。这个成本是隐形的——开发者看到的是「编译好慢」，看不到是哪几行 `#include` 造成的。

现有工具分别回答了这个问题的一段，但都不回答「删掉这一条，这次构建具体省多少时间、多少内存，这个差异是不是噪声」——要么没有反事实，要么只有二值判断。 HeavyHeader 把全部工程力量押在这一段，并把它命名为反事实构建实验（Counterfactual Build Experiment，CBE）：让真实编译器报告自己做了什么，对候选做交替 A/B 真实编译，用越过噪声线的实测差值给出「建议删除 / 保留 / 需人工判断」三态结论。

2.2 竞品边界：每个工具答到哪一句为止

下表是本项目 2026 年 10 月对同类工具的逐项核读（事实核实日期与逐格证据等级见 `docs/competitors.md` 与 `docs/competitor-matrix.md`）。对比的重点不是功能有无，而是每个工具答到哪一句为止。

工具（出处年份）	回答什么问题	停在哪一句（答不了什么）
Header Hero（2011）	谁可能贵：归因行数排序 + Blowup Factor	只排序；正则解析漏宏与条件编译；不验证、不量化删后收益
ClangBuildAnalyzer	现在谁花了多少：逐头文件累计 ms（真实 -ftime-trace）	只描述现状，没有「删了会怎样」；仅 Clang；无基线、无门禁
CompileScore	同上 + 可视化：每 TU 时间线、includers 图	同上；无反事实、无决策、无门禁；Windows / VS 为主，无行数归因
MSVC C++ Build Insights	MSVC 侧的官方答案（含 PCH 该放什么）	同上；删后量化、三态决策、按实测净收益的门禁均不提供
IncludeGuardian	该改什么结构：七类建议（guard、PCH、拆分…）	不编译、不实测（token 口径估算）；无三态；仅 Win/Linux 二进制
deheader（2010）	删了还能编吗：注释掉一条 include 再编译	二值结论，不量化收益；整段跳过 #if 区域；无收益阈值
IWYU / clang-include-cleaner	该不该包含：AST 级必需性判定	不给成本量化；同工程同任务实测 0 条删除建议
Include-Wrangler	每条 include 的边际：省掉它少打开几个文件	只数文件、不测时间（2012 年，已休眠）；自写扫描器，不碰编译器
clang-build-profiler	include 树逐边解析耗时 + 模板实例化树	纯呈现：不算「删了省多少」；无验证、无决策、无门禁

工具（出处年份）	回答什么问题	停在哪一句（答不了什么）
include-what-costs	逐头文件实测编译耗时与内存占用	节点级而非逐条语句（LHCb 出品）；无三态结论，无基线门禁
HeavyHeader（本项目）	删了的净收益是多少、有多大把握：反事实实验 + 三态结论 + 门禁	重载决议选中的模板依赖（AST 区）文本层无信号，由 verify 实测兜底

表中「不提供 / 没有 / 只支持」等判断的依据，是仓库 `docs/competitor-matrix.md` 里带证据等级的逐格核读（D 官方文档 / C 源码核验 / L 本地实验，核实日期 2026-10-18）；未列入该矩阵的两行（clang-build-profiler、include-what-costs）来自 `docs/competitors.md` 的文档级核读，并已于 2026-10-21 对照两个项目的官方仓库复核：前者仍是纯呈现，后者仍是节点级。以上是截至核实日期的检索结论，不是对工具能力永久边界的断言。

同任务对照（说我们更强之前，先跑一个真竞品）：对 clang-include-cleaner（LLVM AST 级，业内标准做法）在 nlohmann/json 64 个编译单元上，它给出 0 条删除建议、101 条补充建议；我们把 222 个候选逐条真实编译，得到 1 条实测 −43.5% 的删除建议。两个同时成立的事实：它分析失败的那一个单元恰是我们实测到 −41.4% 的那个——它的「0 条删除」部分源于没看到这个单元；而它给出的 101 条补充建议我们一条也给不出。对 ClangBuildAnalyzer：同一批 25 个编译单元、同一候选筛选规则，Top-10 重合 7/10、Spearman ρ ≈ 0.68——这组数字支持「中度一致」，不支持「我们更准」。结论是互补而非胜出。

2.3 核心方法：一次编译，三个问题

CBE 的关键是**同一棵源码树、同一条编译命令**跑两遍（一遍原样、一遍删掉候选行），让编译器自己回答三个不同强度的问题——本项目称之为三条真值，由弱到强，任何一条不过，结论就往下降一档：

真值	问什么	怎么测	能证明什么 / 不能证明什么
① 能编过	删掉这一行，语法上还成立吗	真实编译（默认 `-fsyntax-only`，可升为完整编译）	能：这一行不是必需的声明来源；不能：证明完整语义等价（反例见 3.4）
② 少解析多少	删掉它，编译器少读几个文件	两侧 `-H` 输出求差	能：成本确实下降了；不能：下降等于耗时下降
③ 汇编等价	生成的代码还是同一个吗	两侧各编到汇编、归一化后比指纹	能：提供代码生成一致性的强证据；不能：证明完整语义等价；不能：收益一定值得

第三条真值是本方法与「注释掉再编译」类工具的分界：编得过不等于代码生成一致。它的证明范围受编译器、编译选项与指纹归一化规则限制，因此本工具把它当作保守验证门槛，而不是通用形式化证明——这也是三档结论里最高一档措辞的边界。人工评审默认不会做第三轴，所以它拦下来的东西往往正是人眼放过去的东西（§3.4）。

第三条真值：什么时候跑、代价多大。它跑在探针阶段、计时之前——先把「删掉这一行」的版本真编一遍，只有编过的候选才配做比对（编不过的直接判 compile-error），再两侧各编到汇编、归一化后比指纹。顺序不能反过来：反过来就是把被破坏的程序拿去计时、把计时数字当收益（§3.4）。

不是所有候选都跑：`verify` 默认开（`--no-check-codegen` 可关），批量初筛 `scan --probe` 默认关（`--check-codegen` 可开）——初筛要的是排序，不是结论。但两类候选强制跑，开关说了不算：疑似「被包含时执行」的副作用，以及实现片段的一次实例化（删掉通常只是少一个函数定义，编译照样过）。

代价不承诺可忽略：每个候选两侧各多一次编到汇编。实测——`verify --top 3 --runs 2`（demo 语料）12.2 → 13.6 s（+12%）；`scan --probe-compile --probe-top 16`（doctest 24 TU）16.2 → 21.1 s（+30%，+0.31 s/条）——初筛每条本来只编一次，多加两次占比更大，这就是它不在初筛默认开的原因。MSVC 无对应实现（`-S` / `-g0`），闸门直接不跑，报告也不许声称做过校验。

2.4 收益怎么算：独占闭包、交替测量与强制复测

独占闭包：删掉这条 include 后，只有它会停止被解析的头文件集合。如果某个头还能从别的 include 到达，删掉当前这条并不省钱。

交替 A/B 与噪声线：同一候选两侧交替编译、各取 min-of-N，差值必须越过噪声线才计入收益。实测这台机器的单侧读数离散度中位是 11.8 ms，而确认收益只有 2.7–8.8 ms——收益是 min-of-N 压出来的，不是一次读数。强制复测：improved 必须再独立测一遍仍成立才算数（实测复测存活率 73%）。三态判定：建议删除 / 保留 / 需人工判断——旧版把「疑似未使用」同时用来表示「值得试」和「不值得评审」，三态用一个写明阈值的倾向把它们分开。

2.5 消融表：为什么这不是「堆叠现成工具」

拿掉任何一层，都有实测记录在案的错误结论在等着（复现脚本均在 `experiments/` 下）：

拿掉哪一层	会发生什么（实测）	记录在哪
独占闭包计算	候选排序被「所有单元共用的头」占据，排第一的删掉必然编译失败（13 个已知失败候选被压到第 27 名以后）	2.4 edge_closure_check.py
交替测量（A/B interleave）	2.005% 的机器漂移被整块报成优化收益	2.4 aa_noise.py
强制复测	第一轮 4 条 improved 复测 0/4 复现	2.4 repeat_probe.py
三态决策层	用户手里会剩下 187 条「疑似未使用」的假阳性	2.4 failure_taxonomy.py
第三条真值（汇编指纹）	spdlog −77.8% 的「优化」会被当成收益上报（§3.4）	2.3 corpus_timing.py

三、实验与真实工程验证

3.1 先说口径：三类收益不能混

本文档出现三种「收益」，它们是不同指标，互不能代替、也互不能印证。引用时必须带口径。

指标	含义	代表数字	常见误读
完整构建 CPU 降幅	整个工程一次全量构建的 CPU 时间变化（含 PCH 等结构性改动）	292 → 259 s（−11.4%）	不是删一条 include 得到的
单条 include 计时收益	只删这一行、其余不变，编译耗时变化	doctest −3.01%、zlib −2.125%、zstd −46.5%	不是准确率（见 3.5）
解析行数减少	编译器少读多少行头文件文本	zstd 单条 184 → 32 个文件	行数不是秒数（ρ ≈ 0.68）

3.2 完整构建对照：加速的前提是没改变语义

在 nlohmann/json v3.11.3 的测试树上（63 个编译单元），工具一次扫描指出「5 个头文件构成 PCH 候选、覆盖 63/63 个编译单元（估算）」，按此实施后：完整构建墙上时间中位数 43.2 s → 40.1 s（−7.1%），编译 CPU 时间 292 s → 259 s（−11.4%）。

正确性门禁：63 个测试套件逐个运行，断言通过数基线 4,264,414 / PCH 后 4,264,414，逐套件一致，少一条都不算通过。估算与实测分行列出、不相加：估算说「消掉 91.2% 的头文件解析量」，实测说「编译 CPU 省 11.4%」——差在三处（16 个单元改了配置宏用不了 PCH、头文件解析只占编译时间的一部分、行数不是秒数）。原始记录：`experiments/results/experiment-20261017.md`（复现：`python experiments/control.py`）。注意：这是PCH 结构性改动的收益，与 3.3 的「删一条 include」不是同一件事。

同一份报告也给出这条建议的代价，而且选集合的方式本身是贪心（每次加入能让净收益增加最多的候选头，没有正增益就停），不是取排序后的前几个。改成贪心是因为实测到前者会选错：在一个 10 单元的合成语料上（10 个单元全含 common.h、5 个额外含 special.h，两个头等大），取前 2 个的净节省是 12,048 行、覆盖率 50%，只取 common.h 反而是 13,554 行、覆盖率 100%；json 上同一把尺子，取排序前 5 个要 5 个头才达到 88.4% 的净收益，贪心只放 1 个就达到同样的 88.4%——每多一个头就多一层「必须是每个单元的第一个 include」的约束。代价分两层报：影响面（json 上是 `json.hpp`，63/63 个单元、约 113 s），以及脆弱头清单——闭包里触达小于「使用 PCH 的单元数」的项目头，它们只被一部分单元使用，却让全体陪着重编，报告按放大倍数降序列出并给出规避办法；json 上这份清单为空。归档：`experiments/results/pch-greedy-selection-20261024.md`。

那一节是估算，单位是行。`pch-verify` 做的是另一件事：真的生成一份 `.gch`，逐单元双侧重编、计时。原因是估算看不见三件事，而这三件恰好决定一个 PCH 值不值得上——① 一份 .gch 只能在编译选项完全相同的单元之间共享，差一个 `-D` 编译器就静默退回普通编译（退出码 0、收益恰好为 0）

；② 编译器不采用 .gch 时默认不出声，要靠 `-Winvalid-pch` 在 stderr 里找；③ 有些单元加了 PCH 就编不过。

口径	json 单元测试（2026-10-25）
估算覆盖率（上界）	96%（55/57 个单元包含这些头）
实测采用率（下界）	62%（15/24 个单元编译器真的用了 .gch）
采用口径节省	52.6%（14.8 s → 7.0 s）
全组口径节省	46.4%（含 9 个用不上 PCH 的单元）

24 个单元里 9 个加了 PCH 后编不过——宏状态被 .gch 固化。这与 §3.1 那次完整构建实验踩到的是同一个问题，但那次是靠跑测试看断言数发现的、并靠人工排除了 16/63 个配置宏单元；这次工具自己编译一遍就列了出来。

而这 9 个不必等到编译才知道：源文件第一屏就能看出来——它们都在 `#include <nlohmann/json.hpp>` 之前写了 `#define JSON_TESTS_PRIVATE`。于是判据做成静态的：对每个候选头，看它那一行 include 之前有没有 `#define` / `#undef`，只读源文件。准确率由实测打分：召回 100%、精确 90%（10 个预警里 9 个真的编不过，虚报的那个改的是断言宏、不改变类定义）。报告里的措辞是风险清单，不是失败预言——判据能判断「宏状态会被固化」，不能判断「固化后会不会炸」。

收益分两个口径报，因为失败单元的耗时必然更短（提前退出），每个都比基线短 460–530 ms——那不是变快了。行数与秒之间不做换算，所以不报「预测误差」；两者能对的是覆盖率：估算 96% 是上界，实测 62% 是下界，中间那 34 个百分点就是「照着估算直接上 PCH」会踩到的空。三次独立运行差约 0.9 个百分点。复现脚本 `experiments/pch_verify_json.py`，归档 `experiments/results/pch-verify-json63-20261025.md`。

3.3 单条 include 的收益，以及它怎么进入真实工程

工程	改动	实测依据	状态
doctest	删 prelude.h 未用的 `#include <unordered_set>`	24 单元全部编译通过；聚合 20556 → 19937 ms（−3.01%），复测通过	PR #1212 open
zlib	删 zutil.h 未用的 `#include <stdlib.h>`	9 个受影响单元全部通过；3314 → 3244 ms（−2.125%），复测通过	PR #1344 open
zstd	删 pool.c 的 `#include <allocations.h>`	384 → 205 ms（−46.5%），解析文件 184 → 32；同命令第一遍 384 → 213 ms（−44.5%）	全项目最大单条收益

两条上游 PR 正文都写明验证覆盖到哪里、没覆盖到哪里（下游使用者、其他平台工具链）。状态经 2026-10-23 实时核实：doctest#1212 有一条维护者的礼貌回应（提示先读 CONTRIBUTING），codecov 报告「改动行被测试全覆盖」；madler/zlib#1344 提交后无任何回应。两条均为 open、

未被合并，因此本文只写「已提交」，不写「已被采用」——合并与否由上游维护者决定，结果无论如何都会更新进仓库状态表。

zstd 那条还逼出了一次方法论修正：同一行 include 在 `-DZSTD_MULTITHREAD` 下删了编不过——该不该删是构建配置的属性，不是源码的属性。修法不是放宽判据，而是让「用到」只在活代码里成立；这条能力已产品化为 `configs` 命令（§4.4）。

3.4 反例：看起来很成功的错误优化

spdlog 的 TU 层候选按新排序取前 3 条送计时闭环，跑出全部语料里最大的单条计时收益——删掉 `cfg.cpp` 的 `#include <spdlog/cfg/helpers-inl.h>` 后聚合 566 → 125 ms（−77.8%，三次独立运行复现）。接上第三条真值后发现：删除后的汇编从 10,329 行塌缩到 40 行——cfg.cpp 就是把 `load_levels` 等函数编进库的编译单元，那条 include 提供的是它们的定义；`-fsyntax-only` 照样过（声明在别的头里），链接期才会炸。同一语料的另一条 +2.4%~2.9% 同样是静默改程序。

计时单独庆祝的两条，在第三条真值下全部现形——这不是计时测错了，是计时回答不了「程序还是不是同一个」；反事实实验必须三轴齐上。

brotli 1.1.0（32 TU）给出同方向的批量证据：无 guard 的实现片段展开出 31 个实例化站点，实测 0 条可删（28 条编译失败 + 3 条汇编指纹变了）。对照实验更能说明问题：关掉强制闸门后，那 3 条从 codegen-changed 退化成「无变化（−8 ms）」——被拦下的不是虚假收益，而是看起来无害的破坏性改动。只做 `-c` 不链接的探针，对「少一个函数定义」是静默的。

3.5 采纳率的口径，以及结论怎么分档

「可采纳率」曾经在内部被反复引用，直到被指出一个问题：它既不是准确率也不是召回率。拆开之后，每个数字才有确定的分母：

指标	数值	分母是什么	不能用来回答什么
① 池内采纳率	25%（5/20）	送测池：三真值已证实 ∩ 预估过噪声线	不能当端到端准确率；换测量协议同一池是 40%，成员仅重合 3 条
② 产出速度	52 秒/条	扫描 + 三真值 + 计时漏斗 4 分 21 秒换 5 条（Lua 34 TU）	不能当单条成本（含固定扫描开销）；与静态分析阶段的 4.5 秒/条不同分母
③ 端到端	2.3%（5/222）	全部 222 条 include 边；相对「真能删 69 条」是 7.2%	不能当召回率——真值集本身由本工具生成，它没提出的机会不在分母里

所以「25%」只能读作「约四分之一」，成员会摆动：improved 复测存活率 73%（11 → 8），两遍之间成员只重合 3 条。它证明的是一种有价值的筛选方法，尚不是在不同运行间稳定输出同一批优化的预测器。另有 49 条已证实候选确实少解析 2,374 行，只是预估 0.2–0.3 ms、低于噪声线 3–5 倍——不可说成「工具对它们没有潜在收益」，只能说这台机器上测不出来。

结论因此按证据强度分三档展示（命令 `heavyheader verified` 只列第一档），措辞由测试锁死：

证据档	含义	能说的话 / 不能说的话
已验证	汇编指纹一致 + 省时过噪声线 + 复测成立（本工具采用的保守门槛，非形式化证明）	这一条可以照着改
已证等价	汇编指纹一致，但收益没过噪声线	「代码生成一致」已证；「值不值得删」未证——不许当收益报
仅编译通过	只证明删了能编过，没做等价校验	不许说「代码生成一致」：删掉 <iomanip> 后目标文件从 21,289 字节缩到 10,183 字节，能编过但已是另一个程序

3.6 失败与边界（同等重要）

事实	数字	说明
起点很差，且没有被打磨掉	三个工程可采纳率 1/16、1/16、0/16（首轮协议）	被当作迭代起点：定位根因、四轮精度专项、同协议重跑，最终 1/13 且真优化排第 1 位
json 上至今仍是 0	25 个编译单元，0 条可删	没有藏起来——不是每个工程都有可删的 include
闸门放行的候选仍有一半编不过	doctest 16 条里 8 条编译失败	这是编译失败率，与「疑似未使用」判定的假阳性（三语料为 0）不是同一个指标
文本判定的已知边界（AST 区）	一个格式化 std::vector 的候选被判可删、实测编译失败	收益由重载决议选中的模板实例化提供，源码里没有标识符唯一指向这个头
本轮新增收益全在判据放弃的一侧	Lua 两遍各 6 条拿到闸门证据，推荐集贡献 0 条	所以 `verified` 按预估收益排、不按判据排；代价是 20 条里 13 条编不过，命中率低于按判据排的 25%，两者不同分母、不可直接比

3.7 规模换算：仿真检验与 ROI（推算与实测分开报）

前六节的数字都来自这台机器上的真实语料。本节回答一个必然会被问到、而本项目没有客户数据的问题：如果一个更大的工程复现同样的降幅，意味着什么。做法分两步：先用一个成分与 json 对齐的合成工程实测「比例是否随规模稳定」，再把稳定的比例用到假设的 2,000 TU 工程上。2,000 TU 与 60 分钟仍然是假设输入，不是实测。

输入 / 假设	取值	这个取值从哪来
仿真工程规模	2,000 个编译单元	假设输入（真实语料最大 63 TU；仿真实测扫到 500 TU）
一次全量构建的耗时	编译 CPU 时间合计 60 分钟	假设输入；注意是 CPU 时间合计，不是墙上时间
假设可复现的降幅	按区间报（见第二步）	json 63 TU 实测 −11.4%；成分对齐的合成工程 500 TU 实测 −7.4%
全量构建频次	30 次/天（CI 与开发者本地合计）	假设输入

第一步：合成工程实测——比例是否随规模稳定

规模	编译工作量节省	墙上时间节省	墙上/工作量比	有效并行度（−j8）
json 63 TU（真实工程）	11.4%	7.1%	0.62	6.76
合成工程 125 TU	6.0%	5.4%	0.90	7.65
合成工程 250 TU	7.1%	6.9%	0.97	7.89
合成工程 500 TU	7.4%	7.2%	0.97	7.91

合成语料的每个成分都对着 json 归档校准：5 个候选头、25% 的单元因配置宏不能用 PCH、覆盖单元单次编译 1.77 s、PCH 省 22%（json 同构值 21.6%），并且复现了「重单元集中在不能用 PCH 的一侧」这个形状（json 归档里最重的两个单元都在那一侧）。实测给出两条结论：① 收益比例随规模稳定——6.0% → 7.4%，4 倍规模内只差 1.4 个百分点，且这点差异可由「覆盖工作占比」从 53% 收敛到 59% 完全解释；② 墙上/工作量比收敛到 0.97，不是 json 的 0.62——json 那个比值是 63 个单元下的尾巴效应（最重的单元恰好不能用 PCH，压在关键路径上），规模一大就被摊平。原始记录：`experiments/results/scale-sim-2026-10-23.md`。

第二步：把比例用到假设的 2,000 TU 工程上

按上述假设推出的结果	json 路线	仿真路线
每次全量构建省下的编译 CPU	60 × 11.4% ≈ 6.84 min	60 × 7.4% ≈ 4.4 min
折算成墙上时间	6.84 × 0.62 ≈ 4.2 min	4.4 × 0.97 ≈ 4.3 min
CI 机时（30 次/天、22 个工作日）	≈ 3.4 h CPU/天，75 h/月	≈ 2.2 h CPU/天，48 h/月
开发者等待（按墙上时间）	≈ 2.1 h/天	≈ 2.1 h/天

五条必须和上面这张表一起读的限定：

① 2,000 TU、60 分钟、30 次/天都是假设输入，本项目没有任何企业客户数据；被实测支撑的是比例，不是这个工程的存在。

② 编译 CPU 按区间报，不报单值：仿真没有支撑 json 的 11.4% 可以外推——成分对齐的合成工程在同一把尺子下是 7.4%。差距不在规模，在头文件构成（json 的 doctest.h 与 json.hpp 以声明和宏为主，解析占比更高）。所以 6.84 min 是这个区间的上界，不是预期值。

③ 墙上时间 4.2 min 是两条独立路径的交集：json 路线 11.4% × 0.62 与仿真路线 7.4% × 0.97 都落到 4.2–4.3 min。这是全表唯一不依赖「选哪个比例」的数字。

④ CPU 时间 ≠ 墙上时间，且两者的比随规模变大：63 个单元时 0.62，500 个单元时 0.97。拿 json 的 0.62 外推更大的工程会低估墙上收益。

⑤ 不换算成金额：CI 机时单价随供应商、机型与并发数变化，给出金额需要来源，本文没有，留给读者按自有单价换算。

四、工程化与差异化能力

第二章之后的内容是**在核心方法之上长出来的平台能力**。它们各自保留一个最有说服力的案例，其余细节在完整版与仓库文档里。

4.1 差异化能力一览

能力	一句话	证据
成本考古 blame	门禁报警之后，回答「这是谁干的、哪一行引入的」	把成本增量归因到具体 commit 与 include 行（§4.2）
边级归因 edge-cost	从「哪个头贵」到「哪一行贵」：每条 include 边的边际解析成本	一次真编译同时开 -ftime-trace 与 -H，按独占闭包逐边求和（§4.3）
按配置求值 configs	同一棵树按多套构建配置各扫一遍，只报判定变了的候选	zstd 三配置 90/91/93 个候选、仅 3 条判定不同（§4.4）
模板实例化成本归因	把 GCC/MSVC 的 -ftime-report 阶段成本按头文件摆出来	nlohmann/json 25 TU：模板实例化占构建时间 23%–32%（两次扫描），15–20 个单元超过 25%
PCH 收益估算 / 前向声明建议（均带实测闭环）	从 include 图算出 PCH 该放什么，并报出维护税（改一次要让多少单元重编、约多少秒）；给出可前向声明的类型清单。两条估算各有配套实测：`pch-verify` 真的生成 .gch 并重编，`forward-verify` 真的换成前向声明再重编。另有 `iwyu` 与 include-what-you-use 对位：它判正确性、我们判成本	§3.2 的 PCH 候选即由它选出（估算与实测分两张表）；前向声明在三个真实语料上候选数为 0，已如实归档
报告与可部署性	Markdown / HTML / JSON 三格式，HTML 自包含单文件；运行时零第三方依赖；附 `cmake/HeavyHeader.cmake`，可作为一个构建步骤（扫描与门禁两个目标，默认不挂进 ALL）	零依赖由测试守护：新增依赖会让 CI 直接失败；CMake 集成在装了 cmake 的环境里由测试真跑一遍，缺 cmake 则跳过而不假装通过

4.2 成本考古：门禁报警之后，谁干的

`blame` 把「哪个头变贵了」追到是哪一次提交、哪一行 include：对相邻两次扫描做差，按新增/删除的 include 边归因，新增头按最后一次修改它的提交定责。预测与实测分行列出、不相加——对不上就进「未解释」。这条能力做过检索（GitHub 仓库检索 + 项目官网 + 学术检索，2026-10-19）：相邻的 build-time-blame-plugin 分析 Jenkins 控制台历史、build-blame 剖析单次构建、Bencher 一类只画耗时趋势——把成本增量归因到具体 commit 与 include 行的工具，在已核读范围内未见同能力实现（截至 2026-10-19 的检索结论，非「全球首创」声明）。

4.3 边级归因：从「哪个头贵」到「哪一行贵」

`scan` 回答「哪个头文件贵」（节点级），但开发者能删的是源文件里的那一行（边级）。`edge-cost` 一次真编译同时开 `-ftime-trace` 与 `-H`，trace 的嵌套区间还原逐文件自耗时，每条边按独占闭包求和，再跨编译单元聚合。相关实验记录：`experiments/results/edge-spike-doctest-20261019.md`。

4.4 按配置求值：该不该删是构建配置的属性

判定层本来就按每条编译命令里的 `-D` 求值 `#ifdef`，所以不需要新的分析逻辑：`heavyheader configs` 把同一份 `compile_commands.json` 派生几份、各扫一遍、按（单元, 头, 行）对齐，只报判定变了的候选。实测 zstd 三套配置：90 / 91 / 93 个候选，只有 3 条判定不同——而这 3 条正是「删了编不过」与「删了省 46.5%」的分界。

4.5 治理：把一次结论变成每次提交都要过的尺子

项目	数据	说明
G3 门禁	注入回归 3/3 检出；无回归 3/3 放行、零误报；构建损坏 3/3 拦截	以证实过的基线为尺子；已知盲区：行数不变但编译变慢的回归测不出来
工程开销	扫描开销为纯编译时间的 1.15×（GCC）/ 1.20×（Clang），峰值内存 9.2 MB	三次运行结果完全一致
CI	3 系统 × 4 个 Python 版本 = 12 组测试矩阵 + 代码检查 + 自举扫描 + 覆盖率 + MSVC 证据，共 16 个岗位；2026-10-27 核实最新提交 239f85b：CI 14 个岗位全绿，含 macOS 4 组（此前 macOS 曾多次排队被取消，cancelled 非 failed）	2026-10-19 修掉一个慢性问题：CI 自建立起每个提交都是红的（7 个跨平台测试 + 3 个 semantic 测试在 Linux/macOS 上失败）
测试	759 个单元测试全部通过（另有 7 个依赖特定编译器的用例按环境跳过）	样本优先用真实编译器输出逐字复制；按平台中性编写
依赖	运行时零第三方依赖	由测试守护：新增依赖会让 CI 直接失败（GOVERNANCE.md §2）

五、开源治理、局限与路线图

5.1 交付状态

版本	内容	状态
v0.1	可用的核心：三种工具链解析、成本归因、四个命令	已交付；v0.1.0 于 2026-10-19 发布至 PyPI，2026-10-21 在全新虚拟环境复验 pip install 可用
v0.2	从工具到平台：趋势看板、增量扫描、PR 评论机器人、多编译器横评	已交付
v0.3	精确模式：独占闭包判定、可选 AST 交叉检查、前向声明建议、PCH 收益评估	已交付
v0.4	精度引擎：成本考古（blame）、边级归因（edge-cost）	已交付
v0.5	生态集成：CMake、Bazel/Meson、VSCode 插件、IWYU 输出互转	进行中（PyPI 上架已于 v0.1 完成）

5.2 已知局限（不藏在附注里）

局限	影响	现状
AST 区无信号	重载决议选中的模板依赖，源码里没有标识符唯一指向那个头	verify 的实测拦截是最终防线；已如实列出误判样本
单机单编译器可迁移性	第四章数字来自 GCC 16.2.0 (MinGW-w64) + Python 3.13	换编译器版本数字会变、方向不变；CI 三编译器横评持续盯着
门禁盲区	行数不变但编译变慢的回归测不出来	已写入文档；需要时序型指标才能覆盖
真值集自证	端到端召回率的真值集由本工具自己生成	所以只报「池内采纳率」与「端到端发现数」，不报召回率
增量构建税未校准	预测比实测偏高约 1.7×	报告里标为「实验性，勿据此决策」，校准前不作为结论

5.3 Roadmap 与治理文件

Roadmap 的下一批：上游 PR 评审跟进、按头文件预算的治理、GitLab 后端、更多生态集成。`GOVERNANCE.md` 写明决策流程（常规 PR 一人审阅、破坏性变更先开 Issue）与贡献者阶梯。如实说明：仓库目前是个人开发，零外部贡献者、零外部 issue——这些文件是项目采用的结构，不是对已有社区的描述。许可证选 Apache-2.0：对贡献者和使用者都明确、含专利授权条款（对一个要进企业 CI 的工具不是形式问题）。团队为个人申报的开源社区项目，代码仓库、测试、治理文件与基准数据均已随作品材料提交。

5.4 复现入口：每个大数字从哪里来

下表给出本文引用的每个大数字的原始记录、测量环境与复现命令。所有原始记录在 `experiments/results/`，脚本在 `experiments/`，均可用 `python experiments/<name>.py` 重跑。换机器、换编译器，数字会变、方向不变——所以每一行都写了测量环境。

本文的数字	原始记录	测量环境
完整构建 CPU −11.4%、4,264,414 条断言一致	experiment-20261017.md（含 control-20261017.json）	nlohmann/json v3.11.3，63 TU
doctest −3.01%（20556 → 19937 ms）	g2-sites-doctest-20261018.md（含 .json）	doctest 24 TU，g++（MinGW-w64）
zlib −2.125%（3314 → 3244 ms）	g2-confirm-picks-zlib-20261018.json	zlib 15 TU，gcc（MinGW-w64）
zstd 单条 −46.5%（384 → 205 ms、解析 184 → 32）	zstd-verify-20261020b.md	zstd 1.5.6 lib/，39 TU / 351 万行，clang；同命令第一遍 −44.5%
spdlog −77.8% 假收益，汇编 10,329 → 40 行	corpus-timing-spdlog-20261019.md	spdlog cfg.cpp
brotli 31 个实例化站点 0 条可删	instantiation-sites-brotli-20261020.md	brotli 1.1.0，32 TU
门禁 3/3 检出、3/3 放行、3/3 拦截	g3-gate-20261018.md（含 .json）	注入回归的 demo 语料
池内采纳率 25%（5/20）与换协议后的 40%	adopt-rate-timeseries-20261020.md	Lua 5.4.7，34 TU
已验证层级两遍 2 / 3 条、成员零重合	verified-layer-lua-20261020.md（含 .json）	Lua 5.4.7
PCH 估算：闭包 139,180 行、净收益 3,201,140 行（65.8%）	pch-estimate-json25-20261021.md（含 .json）	nlohmann/json 25 TU，g++
模板实例化占构建时间 23.3%–31.7%、15–20 个单元超 25%	benchmarks/results/nlohmann-json-baseline.json	GCC -ftime-report 阶段计时；两次独立扫描，差异来自机器负载

本文的数字	原始记录	测量环境
语料规模：zstd 351 万行 / Lua 109 万行 / brotli 295 万行	zstd-rescan-20261021.md、lua-rescan-20261021.md、brotli-rescan-20261021.md	clang 20；compile_commands.json（clang 20，三份归档各一份）
规模扫描：工作量省 6.0% / 7.1% / 7.4%，比值收敛到 0.97	scale-sim-2026-10-23.md（含 .json）	合成工程 125/250/500 TU，g++ 16.2.0，−j8，每臂 3 次干净构建
汇编闸门开销 +12%（verify）/ +30%（scan --probe）	codegen-gate-cost-20261019.md（在 experiments/ 根目录，不在 results/ 下）	demo 语料（verify）；doctest 24 TU（scan --probe-compile）

复现命令：脚本按 `python experiments/<name>.py` 重跑，`<root>` 换成自己的工程根目录。下面每行对应上表同一行的数字。

```
python experiments/control.py # −11.4%、断言逐套件一致
python experiments/g2_sites.py # doctest −3.01%
python experiments/g2_confirm_picks.py # zlib −2.125%
heavyheader verify <root> --runs 3 # zstd −46.5%
python experiments/corpus_timing.py # spdlog −77.8%、汇编行数
python experiments/instantiation_regression.py # brotli 0 条可删
python experiments/g3_gate.py # 门禁 3/3 × 3
python experiments/repeat_probe.py # 采纳率 25% / 40%
heavyheader verified --top 20 --runs 3 --confirm 2 # 已验证层级
heavyheader scan <root> --pch --pch-top 5 # PCH 估算
heavyheader scan <root> --compiler g++ # 模板实例化占比
heavyheader scan <root> # 语料规模（三份归档）
heavyheader verify <root> --top 3 --runs 2 # 闸门开销：开
heavyheader verify <root> --top 3 --runs 2 --no-check-codegen # 关
python experiments/scale_sim.py --scales 125 250 500 --runs 3 # 规模扫描（约 2 小时）
```

附录：参考资料（实际核读 / 实际使用）

完整版到本版的逐条章节映射（含「移出正文的内容去哪了」与五组硬证据验收清单）见仓库 `docs/restructure-map.md`；精简的原则是转移而非删除——实验过程、逐轮迭代与逐项数据来源都在完整版和仓库里，没有一条被静默丢掉。

同类项目（2026-10-18—19 两轮逐项核读，能力与限制均以官方文档与源码为准）：Header Hero（Bitsquid，2011）、ClangBuildAnalyzer、CompileScore、MSVC C++ Build Insights、IncludeGuardian（v0.0.8）、deheader（ESR，2010）、IWYU / clang-include-cleaner、Include-Wrangler（2012，已休眠）、clang-build-profiler、include-what-costs（LHCb）、build-time-blame-plugin、build-blame、Bencher。逐格证据等级见仓库 `docs/competitor-matrix.md`。

实际使用的编译器文档：GCC 16.2.0（MinGW-w64）的 -H / -ftime-report / -fdump-... 输出；Clang 17+ 的 -ftime-trace；MSVC 的 C++ Build Insights。测试样本里的解析器输出是从上述编译器真实运行逐字复制的。

未引用但想说明的事：本文不引用任何未实际读过的政策条文或论文；「截至指定日期未见同能力实现」一律是基于已核读资料的检索结论，不是「全球首创」的绝对声明。

本作品以 Apache-2.0 许可证开源提交。代码仓库、测试、治理文件与基准数据均已随作品材料提交；本文所有实测数字均可按 5.4 的入口一键重跑。